



Western Norway
University of
Applied Sciences

Chapter 9

Goodrich & Tamassia, Algorithm Design

Text Processing

Sven-Olai Høyland

Document Processing

- › Important function of computers
 - › Edit
 - › Search
 - › Transport over internet
- › Involves
 - › Character strings
 - › Pattern matching



String examples

- › Character strings can come from a wide variety of sources
 - › DNA applications
«CGTAAACTGCTTTTAATCAACGC»
 - › Website
«www.hvl.no»



String terminology

- › P is a string of m characters, $P[0]P[1]\dots P[m-1]$
- › $P[i]P[i+1]\dots P[j]$, where $0 \leq i \leq j \leq m-1$ is a substring of P
 - › Note if $i = 0$ and $j = m - 1$, we have that P is a substring of itself
 - › A proper substring exclude P
 - › Use $P[i\dots j]$ to denote $P[i]P[i+1]\dots P[j]$
 - › If $i > j$, we have the null string / empty string, which has length 0
- › $P[0\dots j]$ is a prefix of P ,
- › $P[i\dots m-1]$ is a suffix of P
- › The null string is a prefix and a suffix of any other string



Empty string in Java

- › `String s = "";` // nothing between ""
- › Has length 0

- › `String s = null;` // means that s is not referring to a string



Problems

- › String P where $|P| = m$
 - › Number of prefixes?
 - › Number of suffixes?
 - › Number of substrings?
 - › Number of substrings that are neither a prefix nor a suffix?



The Pattern Matching Problem

- › Text string T , where $|T| = n$
- › Pattern string P , where $|P| = m$
- › Is P a substring of T ?
 - › Answer: No / starting index of first substring matching P
- › Notation
 - › Σ is character set or alphabet which the character in P and T comes from. Could be a subset of ASCII or Unicode, but is more general.
 - › $|\Sigma|$ = is number of characters in alphabet (fixed constant)



Brute Force Pattern Matching

- › Test all possible placements of P relative to T
- › Begin with P as a prefix of T
- › If P does not match, move P one position to the right



Brute Force Algorithm

Algorithm BruteForcematch (T, P)

Input: String T (text) with n characters and P (pattern)
with m characters

Output: Starting index of the first substring of T matching
P, or an indication that P is not a substring of T

```
for i = 0 to n - m do
```

```
    j = 0
```

```
    while (j < m and T[i+j] = P[j]) do
```

```
        j++
```

```
    if j = m then
```

```
        return i
```

```
return "There is no substring of T matching P"
```



Running time

- › Worst case
 - › For each candidate index, we may perform up to m comparisons
 - › There are $(n - m + 1)$ candidate indices
 - › Total: $O((n - m + 1) m)$ which is $O(mn)$
 - › Example that matches bound (given in lecture / book)



Example

- › T = abacaabacc
- › P = aca



Boyer-Moore

- › Usually avoid examining every character in T (which might come as a surprise)
- › Assume alphabet is of fixed, finite size. (Not necessary for Brute force algorithm)
- › Works best when the alphabet is moderately sized and the pattern relatively long
- › Improve Brute-Force algorithm by adding two time-saving heuristics (details next slide)
 - › Looking-Glass Heuristic
 - › Character-Jump Heuristic



Heuristics

- › Looking-Glass Heuristic:
 - › When testing a possible placement of P against T , begin the comparison from the end of P and move backward to the front of P
- › Character-Jump Heuristic: When $T[i] = c$ fails to match character in P
 - › If c is not contained in P , then move P completely past $T[i]$
 - › Otherwise, shift P until the last occurrence of character c in P gets aligned with $T[i]$ (but we never move P to the left, always at least one position to the right)



BM, the function last(c)

- › Use to jump
- › last(c)
 - › If c is in P, then it is the index (starting from 0) of the last (right-most) occurrence of c in P
 - › Otherwise, it is defined as -1
- › Example: «there»
- ›

c	e	h	r	t	a (and all others)
last(c)	4	1	3	0	-1



Boyer-Moore Algorithm

Algorithm BMMatch(T, P)

Input/output: As in BruteForce algorithm

compute the function last

$i \leftarrow m - 1$

$j \leftarrow m - 1$

repeat

if $P[j] = T[i]$ then

if $j = 0$ then

return i

else

$i --$

$j --$

else // jump step

$i \leftarrow i + m - \min(j, 1 + \text{last}(T[i]))$

$j \leftarrow m - 1$

// at the end of the pattern

until $i > n - 1$

return "There is no substring of T matching P"



Example

- › T = «a pattern matching algorithm»
- › P: «rithm»
- › Last-function

c	h	i	m	R	t	others
last(c)	3	1	4	0	2	-1



Example (search for «rithm»)

To be filled in during lecture

a		p	a	t	t	e	r	n		m	a	t	c	h	i	n	g		a	l	g	o	r	i	t	h	m
---	--	---	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---	---



Example (answer)

a	p	a	t	t	e	r	n	m	a	t	c	h	i	n	g	a	l	g	o	r	i	t	h	m		
r	i	t	h	m																						
	r	i	t	h	m																					
					r	i	t	h	m																	
									r	i	t	h	m													
														r	i	t	h	m								
																			r	i	t	h	m			
																						r	i	t	h	m

- › Red indicates comparisons



Running time

- › $|T| = n$
- › $|P| = m$
- › Compute last function: $O(m + |\Sigma|)$
- › Searching: $O(mn)$
- › Total: $O(mn + |\Sigma|)$
- › Worst case
 - › $T = \text{«aaa...a»}$, $P = \text{«baaa...a»}$
- › Experiments
 - › English text where we look for five-character pattern $\rightarrow 0.24$ comparisons per text character
- › Not so good for binary strings or very short patterns



Knuth-Morris-Pratt

- › Main idea
Preprocess the pattern P to compute a failure function f that indicates the proper shift of P (reuse previously performed comparisons)
- › Running time
 $O(m + n)$ which is optimal



Failure function

- › $f(j)$ is defined as the length of the longest prefix of P that is a suffix of $P[1..j]$
 - › $f(0) = 0$
- › «Encodes» repeated substrings inside the pattern itself
- › Example: «abacab»

j	0	1	2	3	4	5
P[j]	a	b	a	c	a	b
f(j)						



Failure function

- › $f(j)$ is defined as the length of the longest prefix of P that is a suffix of $P[1..j]$
 - › $f(0) = 0$
- › «Encodes» repeated substrings inside the pattern itself
- › Example: «abacab»

j	0	1	2	3	4	5
P[j]	a	b	a	c	a	b
f(j)	0	0	1	0	1	2



Knuth-Morris-Pratt Algorithm

Algorithm KMPMatch(T, P)

Input/output: As in BruteForce algorithm

$f \leftarrow \text{KMPSuccessFunction}(P)$

$i \leftarrow 0$

$j \leftarrow 0$

while $i < n$ do

 if $P[j] = T[i]$ then

 if $j = m - 1$ then

 return $i - m + 1$

$i \leftarrow i + 1$

$j \leftarrow j + 1$

 else if $j > 0$

$j \leftarrow f(j - 1)$

 else

$i \leftarrow i + 1$

return "There is no substring of T matching P"



> Searching for «abacab» in text in heading

a	b	a	c	a	a	b	a	c	c	a	b	a	c	a	b	a	a	b	c
a	b	a	c	a	b														

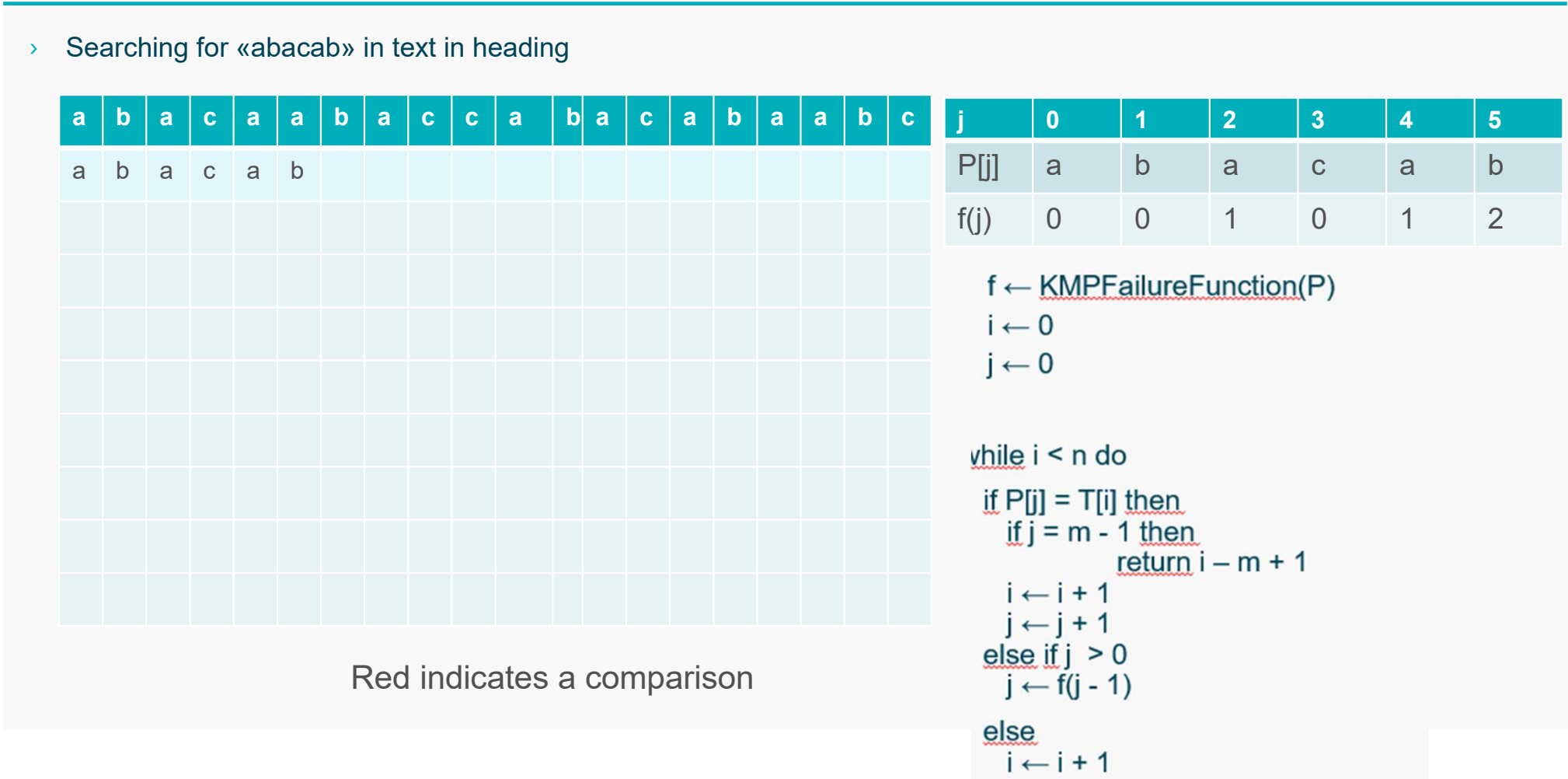
Red indicates a comparison

```

j      0    1    2    3    4    5
P[j]   a    b    a    c    a    b
f(j)   0    0    1    0    1    2

f ← KMPFailureFunction(P)
i ← 0
j ← 0

while i < n do
  if P[j] = T[i] then
    if j = m - 1 then
      return i - m + 1
    i ← i + 1
    j ← j + 1
  else if j > 0
    j ← f(j - 1)
  else
    i ← i + 1
  
```



Example (answer)

a	b	a	c	a	a	b	a	c	c	a	b	a	c	a	b	a	a	b	c
a	b	a	c	a	b														
				a	b	a	c	a	b										
					a	b	a	c	a	b									
									a	b	a	c	a	b					
										a	b	a	c	a	b				
										a	b	a	c	a	b				

Red indicates a comparison



Performance Knuth-Morris-Pratt Algorithm

Algorithm KMPMatch(T, P)

Input/output: As in BruteForce algorithm

$f \leftarrow \text{KMPSFailureFunction}(P)$

$i \leftarrow 0$

$j \leftarrow 0$

Define $k = i - j$

while $i < n$ do

 if $P[j] = T[i]$ then

 if $j = m - 1$ then

 return $i - m + 1$

$i \leftarrow i + 1$

$j \leftarrow j + 1$

 else if $j > 0$

$j \leftarrow f(j - 1)$

 else

$i \leftarrow i + 1$

return "There is no substring of T matching P"



Performance

- › Excluding the computation of the failure function, the running time is proportional to number of iterations of the while-loop
- › Let $k = i - j$. Have $k \leq n$. Three cases occurs in each iteration
 - › If $T[i] = P[j]$, both i and j increases by 1 so k is unchanged
 - › If $T[i] \neq P[j]$ and $j > 0$, then i does not change and k increases by at least 1 (since j decreases by at least 1)
 - › If $T[i] \neq P[j]$ and $j = 0$, then i increases by 1 and k increases by 1 (since j does not change)
- › In each iteration of the loop, either i or k increases by at least 1
- › Hence the total number of iterations of the while-loop in KMP is at most $2n$



Computing failure function

(sufficient to know that there is an efficient algorithm)

- › Algorithm `KMPFailureFunction(P)`
- › Input: String `P` (pattern) with `m` characters
- › Output: The failure function `f` for `P`, which maps `j` to the length of the longest prefix of `P` that is a suffix of `P[1..j]`
 - › $i \leftarrow 1$
 - › $j \leftarrow 0$
 - › $f(0) \leftarrow 0$
- › while $i < m$ do
- › if $P[j] = P[i]$ then
 - › $f(i) \leftarrow j + 1$
- › $i \leftarrow i + 1$
 - › $j \leftarrow j + 1$
- › else if $j > 0$
 - › $j \leftarrow f(j-1)$
- › else
- › $f(i) \leftarrow 0$
- › $i \leftarrow i + 1$



Performance (1)

- › To compute the KMPFailureFunctions takes time $O(m)$.
 - › Analysis is analogous to that of algorithm KMPMatch



Performance (2)

- › Theorem: The Knuth-Morris-Pratt algorithm performs pattern matching on a text string of length n and a pattern string of length m in $O(n + m)$ time.
- › Time is proportional to read the strings T and P separately
- › Notice: The running time does not depend on the size of the alphabet



Standard tries

- › Let S is a set of string from alphabet Σ , such that no string in S is a prefix of another string. A standard trie for S is an ordered tree T with the following properties
 - › Each node of T , except the root, is labeled with characters of Σ .
 - › The ordering of the children of an internal node of T is determined by a canonical ordering of the alphabet Σ .
 - › T has s external nodes, each associated with a string of S , such that the concatenation of the labels of the nodes on the path from the root to an external node v of T yields the string of S associated with v .



Important

- › Have assumed that no string in S is a prefix of another string
 - › Ensures that each string is uniquely associated with an external node in T
- › No limitation since if the assumption is not true originally, we can add a special symbol to the alphabet to terminate each string



Construction

- › Insert one string at a time
 - › Try to trace the path associated with X in T .
 - › Will end in an internal node v since no string is a prefix of another string
 - › Create a new chain of node descendants of v to store the remaining characters of X .
- › Performance:
Time to insert X is $O(dm)$, where m is the length of X and d is the size of the alphabet.
Entire trie for set S takes $O(dn)$, where n is the total length of the strings of S

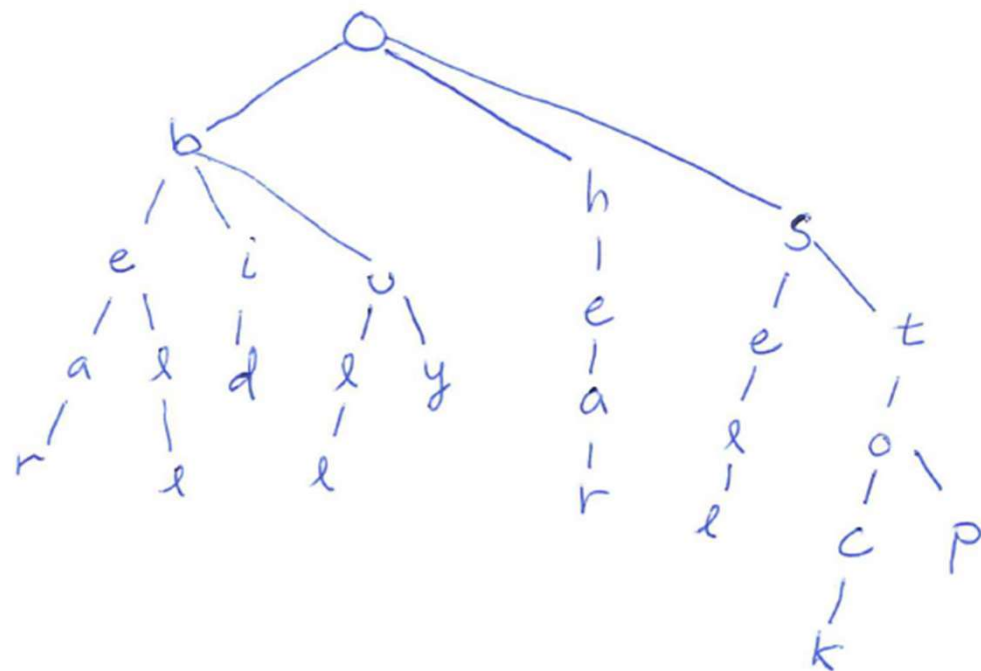


Example (shown in classroom)

› $S = \{\text{bear, bell, bid, bull, buy, hear, sell, stock, stop}\}$



Solution



Theorem

- › A standard trie storing a collection S of s strings of total length n from an alphabet of size d has the following properties:
 - › Every internal node of T has at most d children
 - › T has s external nodes
 - › The height of T is equal to the length of the longest string in S .
 - › The number of nodes of T is $O(n)$.



Performance

- › Worst case for the number of nodes of a trie
 - › Does not exist a pair of strings with a nonempty common prefix
 - › All internal nodes will have one child
 - › But notice that we can not have more than d strings (otherwise at least two must have a common prefix)



Performance

- › Running time
 - › Takes $O(dm)$ to search for a string of size m where d is the size of the alphabet



Example of use

- › Word matching
 - › Want to determine whether a given pattern matches one of the words of the text exactly
- › Dictionary



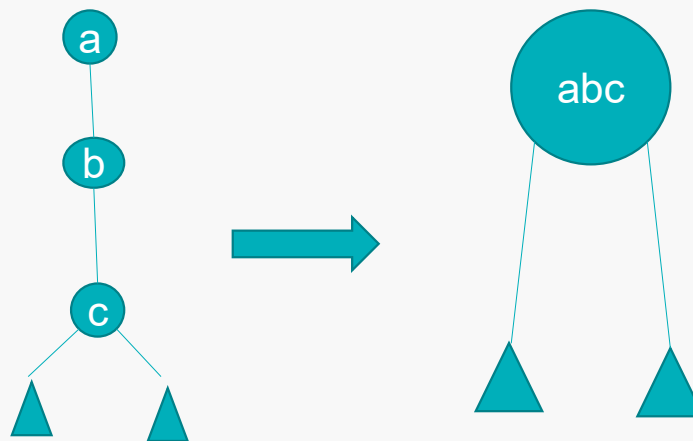
Problem with tries

- › Space inefficiency
 - › May have lot of nodes that have only one child
- › Will look at compressed tries also known as Patricia trie
 - › (Acronym for "Practical Algorithm To Retrieve Information Coded In Alphanumeric")



Compressed tries

- › Similar to standard tries, but ensures that each internal node in the trie has at least two children
 - › Done by using substrings to label nodes rather than individual characters
 - › Replace chains of single-child nodes, with substrings
 - › Example



Compressed tries

- › Advantage
 - › Number of nodes is proportional to the number of strings and not to their total length
 - › Applies when it is used as an auxiliary index structure over a collection of strings already stored in a primary structure



Example, array of strings

Logical order	Physical order in S	[0]	[1]	[2]	[3]	[4]
7	0	s	e	e		
1	1	b	e	a	r	
8	2	s	e	l	l	
9	3	s	t	o	c	k
4	4	b	u	l	l	
5	5	b	u	y		
3	6	b	i	d		
6	7	h	e	a	r	
2	8	b	e	l	l	
10	9	s	t	o	p	

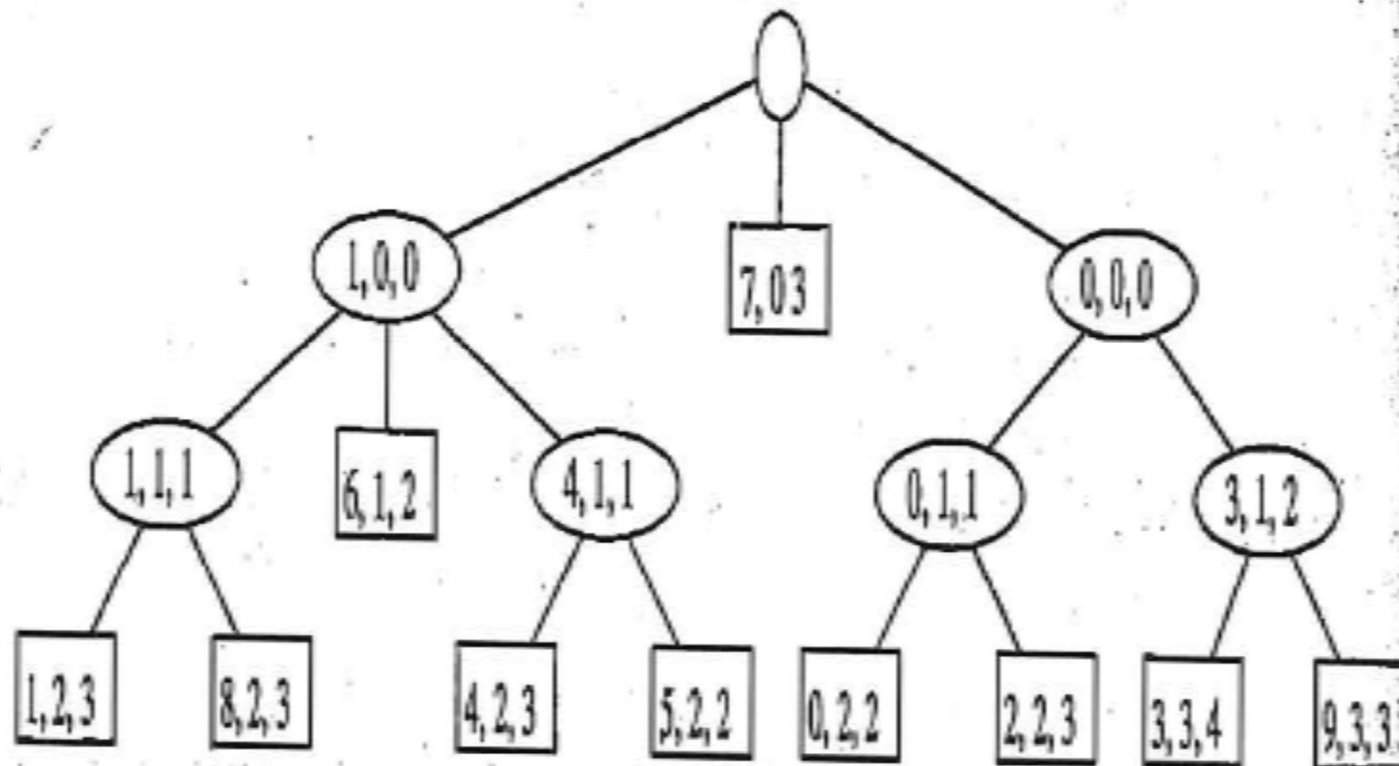


Example

- › Store a triplet of integers (i, j, k) such that $X = S[i][j..k]$. Substring of $S[i]$ consisting of the characters from j to k (included)
- › Shown in lecture (page 434 in textbook)



From textbook, page 434



Properties

- › Theorem: A compressed trie storing a collection S of s strings from an alphabet of size d has the following properties:
 - › Every internal node of T has at least two and at most d children
 - › T has s external nodes
 - › The number of (total, both internal and external) nodes of T is $O(s)$.



Suffix tries

- › The strings in the collection S are all suffixes of a string X .
- › Can simplify representation such that the label of a node is a pair (i, j) indicating $X[i..j]$
- › Add a special character $\$$ that is not in the original alphabet to satisfy that a suffix is not a prefix of another suffix
- › If X has length n , we build a trie for set of strings $X[i..n - 1]\$$ for $i = 0, \dots, n - 1$



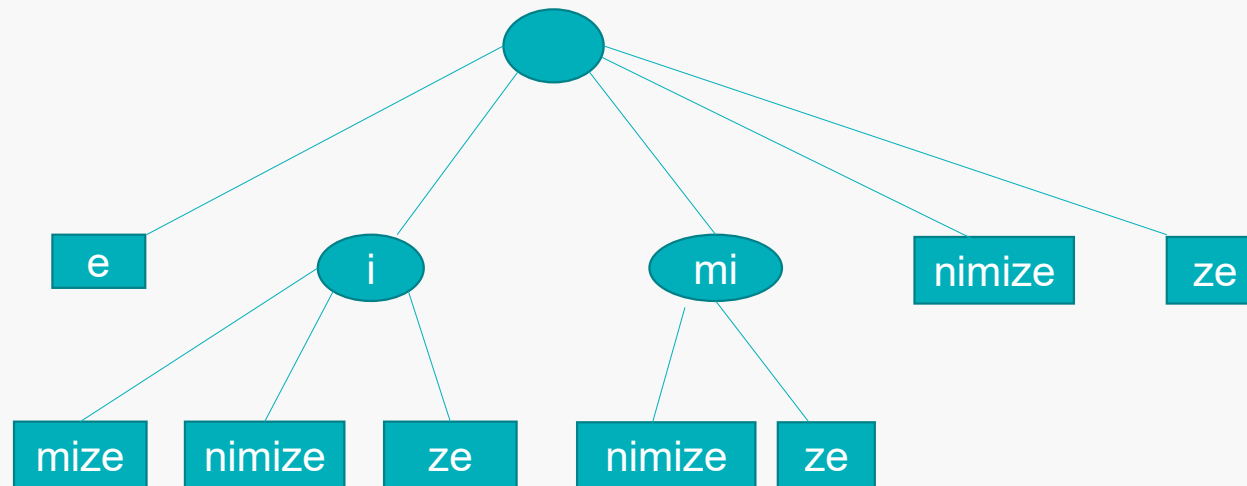
Example (used in classroom, solution next slide)

- › Suffix trie for the string $X = \text{«minimize»}$



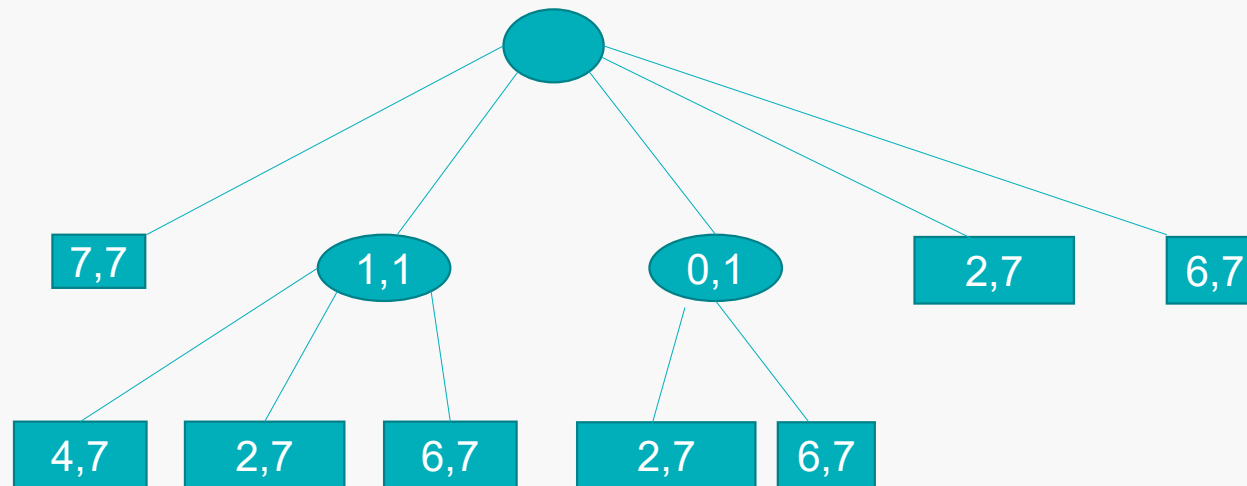
Example

- › Suffix trie for the string X = «minimize»



Example

- › Compact representation of suffix trie for the string $X = \text{«minimize»}$



Construction

- › Can use algorithm for general tries
 - › Takes $O(dn^2)$ because the total length of the suffixes is quadratic in n .
- › However, exists specialized linear algorithm
 - › Fairly complex, but we can consider construction is very efficient



Using a suffix trie

- › Efficient to perform pattern matching queries on text X .
 - › Determine whether a pattern P is a substring of X
 - › Try to trace a path associated with P in T
 - › P is a substring of X if and only if such a path can be traced
- › Why does it work?
- › Works because if P is a substring of X , then we can find P as a prefix of a suffix of X .

- › Name algorithm `suffixTrieMatch`



Performance

- › Running time does not depend on the size of the text X
- › Running time is linear in the size of the pattern
- › Suffix tries are suited for repetitive pattern matching applications
 - › A series of pattern matching queries is performed on a fixed text.



Theorem

- › Let X be a text string with n characters from an alphabet of size d . We can perform pattern matching queries on X in $O(dm)$ time, where m is the length of the pattern with the suffix trie of X . The suffix trie uses $O(n)$ space and can be constructed in $O(dn)$ time.



Search engines

- › Webcrawler
 - › Gather information about webpages
 - › Store information in a dictionary database
- › Web search engine
 - › Allow users to retrieve relevant information from database



Inverted files

- › Also called inverted index
- › Key-value pairs (w, L) where w is a word and L is a collection of references to pages containing word w
 - › The keys are called index terms
 - › L is called occurrence lists



Implementation

- › Efficient implementation
 - › Array storing the occurrence list of the terms
 - › Stored on disk
 - › A compressed trie for the set of index terms, where each external node stores the index of the occurrence list of the associated term
 - › Stored in internal memory



Query

- › Single keyword
 - › Similar to a word matching query
- › Multiple keywords
 - › Retrieve the occurrence list for each keyword using the trie
 - › Return intersection
- › Ranking the pages
 - › By relevance
 - › Important area



Text compression

- › Given a string X over some alphabet such as ASCII or Unicode
 - › Want to efficiently encode X into a small binary string Y
 - › Important for communication over low-bandwidth channel
 - › Storing large documents
- › Will look at Huffman code



Huffman code

- › ASCII and Unicode uses fixed length binary strings to encode character (7 and 16)
- › Huffman code uses variable length
 - › Based on frequencies
 - › Short length for high frequency characters
 - › Long for low frequency characters
 - › For each character c , we have a count $f(c)$ which corresponds the number of times c occurs in X



Prefix code

- › No code word is a prefix of another code word
- › Simplifies the decoding



Huffman algorithm

- › Construct a binary tree
- › Each node except the root represent a bit in a code word
 - › Left child represent 0
 - › Right child represent 1
- › Each external node v is associated with a specific character
 - › Code word for the character is defined by sequence of bits associated with the nodes in the path from the root to the node
- › Frequency
 - › External node – frequency of corresponding character
 - › Internal node – sum of frequency of all external nodes in subtree



Huffman algorithm

- › Let each of the d characters be a binary tree
- › While there are more than one tree
 - › Merge the two trees with smallest frequencies
- › Running time
 - › Each iteration $O(\log d)$ using a priority queue implemented as a heap
 - › Loop is executed d times
 - › Total $O(n + d \log d)$ where n is the length of X



Example

- › $X = \text{«a fast runner need never be afraid of the dark»}$
- › Frequene table (ordered lexicographically)

c	F(c)
blank	9
a	5
b	1
d	3
e	7
f	3
h	1

c	F(c)
i	1
k	1
n	4
o	1
r	5
s	1
t	2

c	F(c)
u	1
v	1



Frequency table ordered by frequency

c	" "	e	a	r	n	d	f	t	b	h	i	k	o	s	u	v
f(c)	9	7	5	5	4	3	3	2	1	1	1	1	1	1	1	1



Encoding/Decoding

- › Encoding of a character
 - › Start in the root and find the path to the character while you write down 0 if you go left and 1 if you go right. From example
 - › a -> 010
 - › v -> 111111
- › Decoding
 - › Process the string from left to right. Start in the root, go left if there is a 0 and right if there is a 1
 - › When you end up in a leaf (external node), write down the corresponding character.
 - › Start again in the root
 - › Example
 - › 10111100111000 -> hei



Text similarity testing

- › Relevant in many situations
 - › Genetics
 - › Long common subsequences in DNA means that two individuals are related
 - › Software engineering
 - › Changes from one version to next
- › Unix
 - › diff command



Longest common subsequence problem

- › Given a string X of size n , a subsequence of X is any string that is of the form $X[i_1]X[i_2] \cdots X[i_k]$, $i_j < i_{j+1}$ for $j = 1, \dots, k - 1$
- › Example
 - › abcdef
 - › Subsequence: bef



Longest common subsequence problem

- › Given two strings X of size n and Y of size m
- › Find the longest common subsequence
- › Enumerate all and take the largest
- › Can do better with dynamic programming



Dynamic programming

- › $X[0..i]$ and $Y[0..j]$
- › Longest common subsequence $L[i,j]$
 - › If $c = X[i] = Y[j]$ then longest common subsequence ends with c
 - › $L[i,j] = L[i-1,j-1] + 1$
 - › If $X[i] \neq Y[j]$ then the sequence can not contain both $X[i]$ and $Y[j]$
 - › $L[i,j] = \max\{L[i-1,j], L[i,j-1]\}$
- › Boundary cases
 - › $L[i,-1] = 0$ for $i = -1, 0, \dots, n-1$
 - › $L[-1,j] = 0$ for $j = -1, 0, \dots, m-1$



Example

- › Y=cgataattgaga
- › X=gttcctaata

L	-1	c	g	a	t	a	a	t	...
-1									
g									
t									
t									
...									

- › Shown in lecture (you find it also in hand outs)



Example (filled in)

- › Y=cgataattgaga
- › X=gttcctaata

L	-1	c	g	a	t	a	a	t	...
-1	0	0	0	0	0	0	0	0	
g	0	0	1	1	1	1	1	1	
t	0	0	1	1	2	2	2	2	
t	0	0	1	1	2	2	2	3	
...									

- › Shown in lecture (you find it also in hand outs)

